Warehouse Inventory & Reorder System



Team No: 15

Course: Problem Solving Using Programming (JAVA)

Submitted To: Ms. Swathi

By: B.Srinitha - 2620030365

Abstract

A Java-based application for efficient **inventory management** and **smarter reorder decisions**.

Real-time inventory tracking

Automated reorder alerts

CRUD operations on inventory

File-based data persistence

Reports for low and out-of-stock items

A robust console-based system that **minimizes stockouts** and streamlines warehouse operations.



Understanding Problem Statement

Context and challenges addressed by the system.

PROBLEM	PROPOSED SOLUTION
Stockouts due to unmonitored reorder levels	Digital catalog with quantity, price, reorder level
Overstocking tying up capital and space	Automated reorder flagging
Data inconsistency from manual records	CRUD operations support
Delayed reordering without timely alerts	File handling for data persistence
Poor reporting capabilities	Summary reports using collections and streams

SCOPE | Console-based application for small to medium warehouses



CO 1 – From Problem to Program

Computational thinking and Java foundations applied to **warehouse inventory & reorder logic.**

Think it through	Break the stock problem into clear steps and design the inventory flow
Code it right	Use Java basics—types, variables, operators, I/O and expressions
Automate the decision	Check stock levels and decide when to reorder
See it in action	A simple Java check for reorder level

```
int id, qty, reorderLevel, reorderQty;
 String name;
 double price;

Scanner scanner =  new Scanner (System.in);
int quantity = scanner.nextInt();

 System.out.println("*** REORDER ALERT: order " + p.reorderQty + " of " +
p.name);
```

CO 2 – Control Flow

Selection & Iteration: Module 2 concepts and their **project applications**.

CONCEPT	APPLICATION IN PROJECT
if / if-else	existence and quantity checks
Nested-if	reorder trigger logic
if-else-if Ladder	stock categorization
Switch-case	menu-driven user interface
while, do-while loops	menu repetition and initialization
for and nested loops	inventory traversal and grouping
continue and infinite loop usage	skip invalid items and continuous operations

```
switch(choice) {
   case 1:  add(); break;
   case 2: receive(); break;
   case 3: issue(); break;
   case 4: view(); break;
   case 5: report(); break;
   default: System.out.println("
                  "Invalid choice!";
}
```

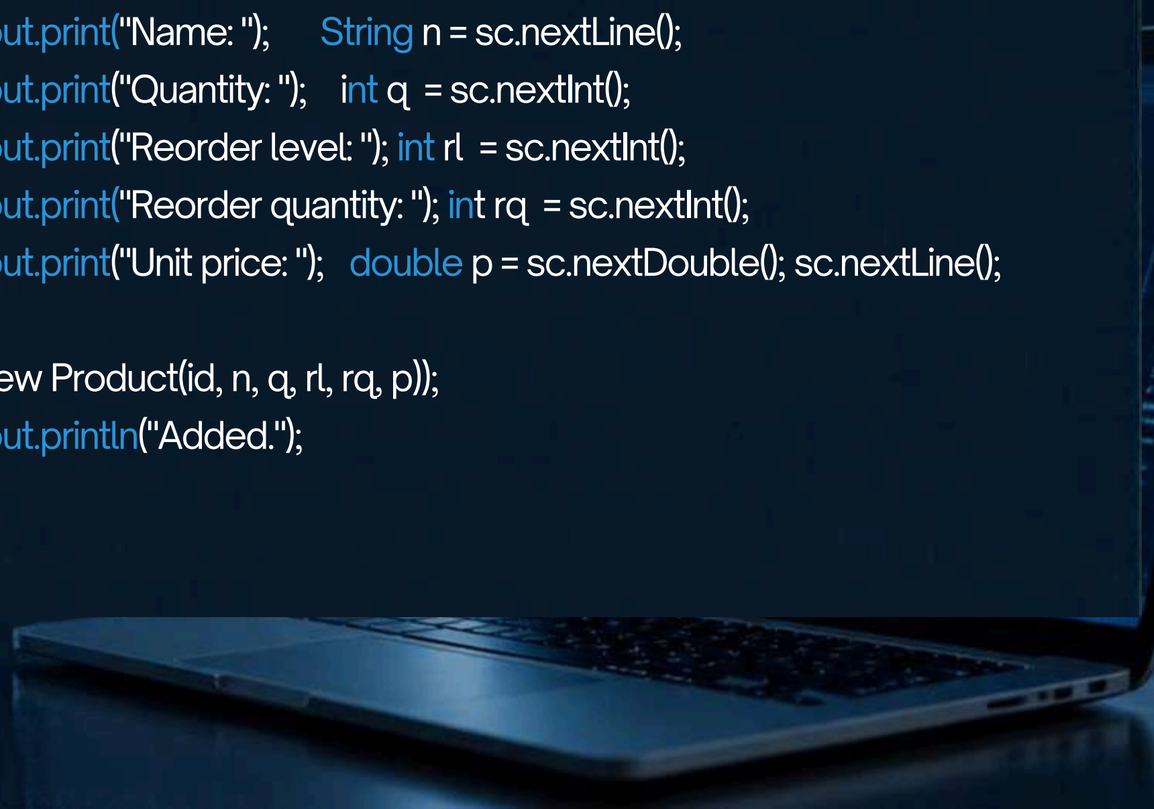
CO 3 – Decomposition with Methods, Recursion & Arrays

- Modular programming with methods for each operation
- Parameter passing and return values in methods
- Method overloading examples
- Enhanced for-each loops
- Recursive binary search with base case

```
static void add() {
   System.out.print("ID: ");       int id = sc.nextInt(); sc.nextLine();
   if (find(id) != null) { System.out.println("ID already exists"); return; }

   System.out.print("Name: ");      String n = sc.nextLine();
   System.out.print("Quantity: ");    int q  = sc.nextInt();
   System.out.print("Reorder level: "); int rl  = sc.nextInt();
   System.out.print("Reorder quantity: "); int rq  = sc.nextInt();
   System.out.print("Unit price: ");   double p = sc.nextDouble(); sc.nextLine();

   list.add(new Product(id, n, q, rl, rq, p));
   System.out.println("Added.");
  }
```



CO 4 – Object-Oriented Modelling, Design & Exception Handling

Module 4 concepts **demonstrated in the project.**

AREA	KEY CONCEPTS
Core OOP	Classes, objects, fields, methods
Construction	Object creation, constructors, constructor overloading
Encapsulation	this keyword usage getters/setters with validation static fields and methods toString() override
Polymorphism & Abstraction	Inheritance and method overriding interfaces for reorder checking
Exception Handling	try-catch finally multic-catch throw / throws

```
public class Item {
    private int itemId;
     private String itemName;
     private int quantity;
     private double price;
    private static int totalItems = 0;

   public Item(int itemId, String itemName, int quantity, double price) {
     this .itemId = itemId;
     this .itemName = itemName;
     this .quantity = quantity;
     this .price = price;
      totalItems+%;


    public int getQuantity() { return quantity; }

   public void getQuantity() {
    public void setQuantity(int quantity) {
      if (quantity < g)
        throw new IllegalArgumentException("Qty cannot be negative");
      this .quantity = quantity;


    @override
     public String toString() {
   return itemId + "" | "" + itemName + "" | Qty: "" + quantity + " | Rs." + price;

}
```

CO 5 – Strings, Library Algorithms & File I/O

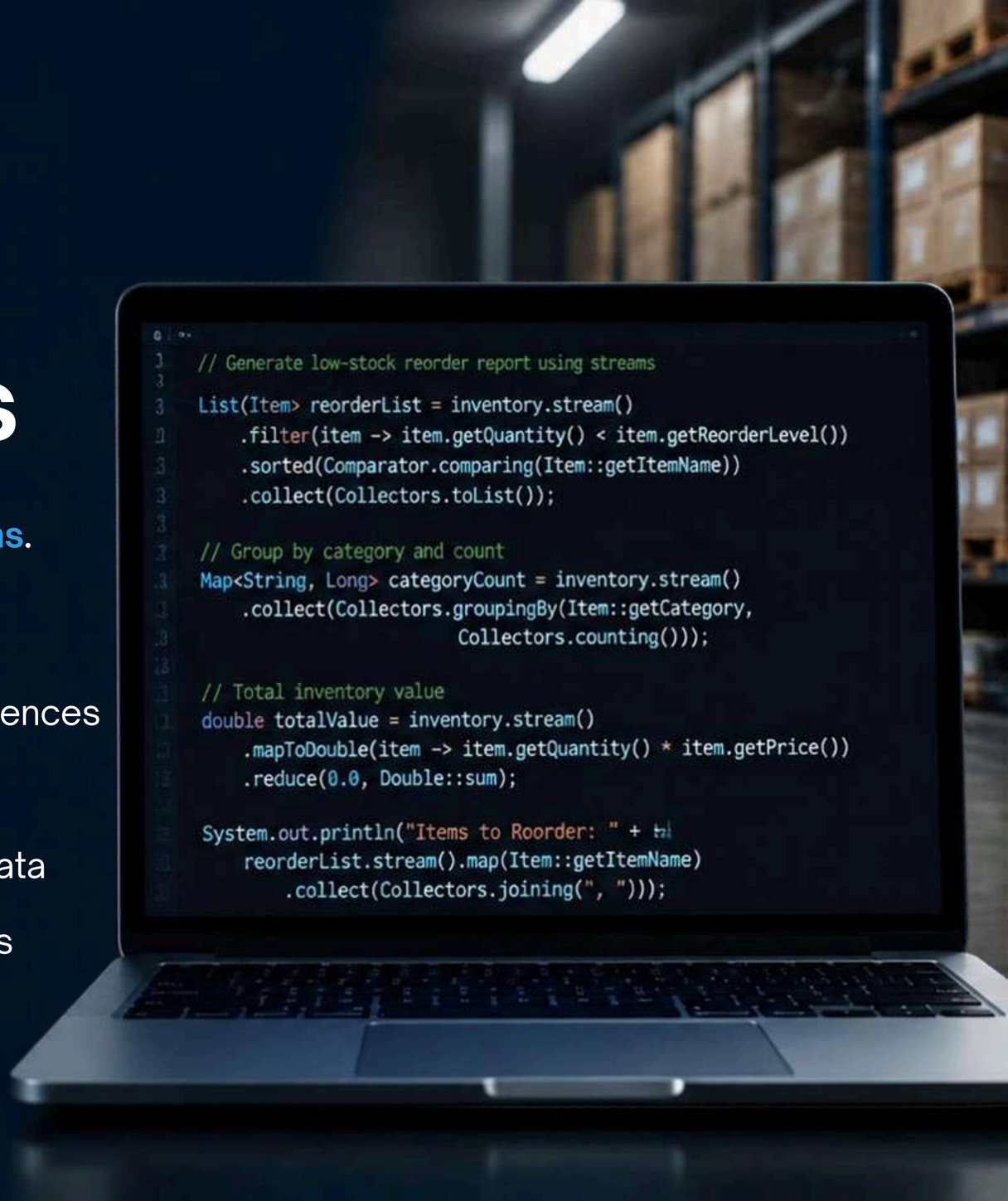
String operations	length(), substring(), indexOf(), equals(), concat, split, join, trim
Data processing	validate and tokenize file data; handle missing files and malformed records
Sorting & comparison	Arrays.sort(), Collections.sort(); Comparable and Comparator interfaces
File input & output	java.nio file, BufferedReader, BufferedWriter for reading and writing files
Resource management	use try-with-resources for automatic stream closing



CO 6 – Functional Programming with Collections & Streams

Build powerful, concise logic with **Java collections** and **streams**.

- **Work with collections** using generics and efficient iteration
- **Express behavior** with lambda expressions and method references
- **Apply functional interfaces** for clean, reusable code
- **Create stream pipelines** to filter, transform, and aggregate data
- **Collect results** with built-in collectors and terminal operations



Conclusion

A complete solution for efficient inventory control and smarter reorder decisions.

KEY TAKEAWAYS

C01	Understand warehouse operations
C02	Manage inventory efficiently
C03	Track inventory in real time
C04	Automate reorder decisions
C05	Store data securely and reliably
C06	Build a scalable, maintainable system

BENEFITS

- Reduces manual errors
- Automated reorder alerts
- Efficient data retrieval
- Persistent storage
- Scalable OOP design

FUTURE ENHANCEMENTS

- GUI with JavaFX or Swing
- Database integration (JDBC + MySQL)
- Barcode/QR code scanning
- Web dashboard with REST API
- Email/SMS reorder notifications



Thank You!

Delivering efficiency, accuracy, and growth for your warehouse.

Screenshot of Output

```
PS C:\Users\DELL\OneDrive\Documents\PSPJ_Proj>  & 'C:\Program Files\Ec
s\DELL\AppData\Roaming\Code\User\workspaceStorage\33fb852a77101952eb37
1.Add  2.Receive  3.Issue  4.View  5.Reorder  6.Exit
Choice: 1
ID: 104
Name: Sri
Quantity: 60
Reorder level: 40
Reorder quantity: 300
Unit price: 5.00
Added.

1.Add  2.Receive  3.Issue  4.View  5.Reorder  6.Exit
Choice: 2
ID: 104
Quantity received: 67
New quantity: 127

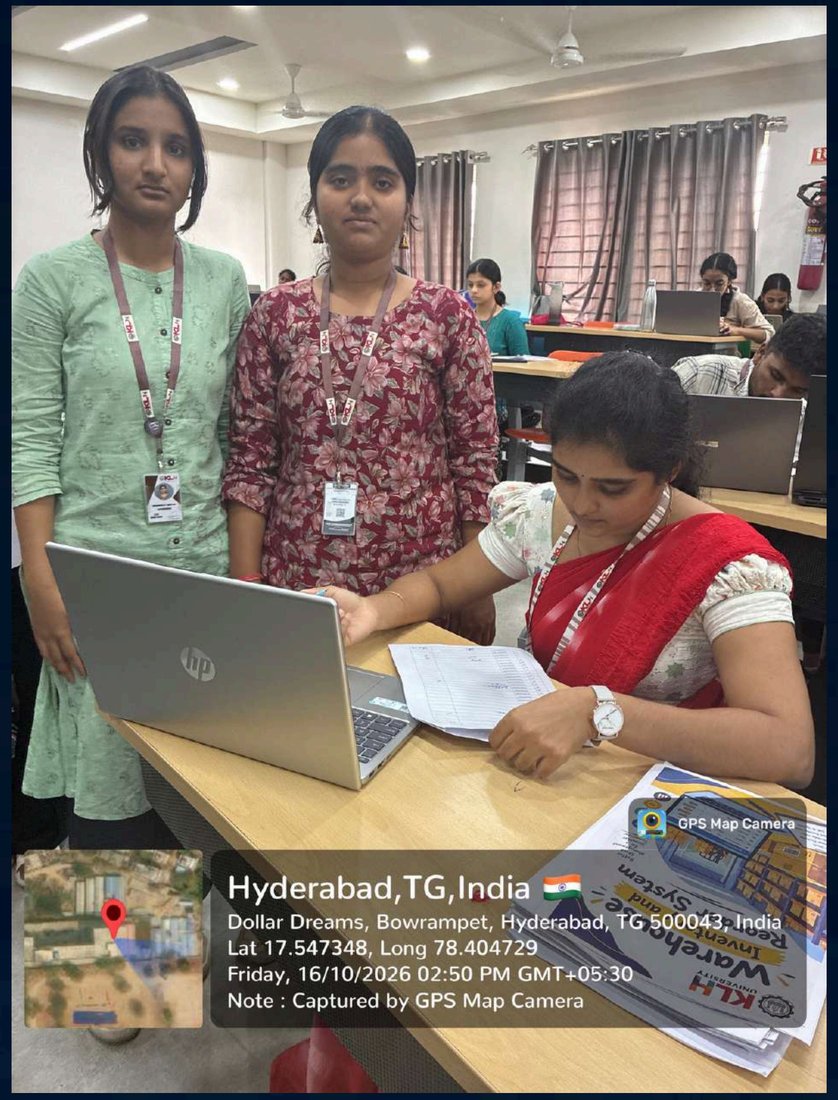
1.Add  2.Receive  3.Issue  4.View  5.Reorder  6.Exit
Choice: 3
ID: 104
Available: 127
Quantity to issue: 50
Remaining: 77
```

```
1.Add  2.Receive  3.Issue  4.View  5.Reorder  6.Exit
Choice: 4
101  Bolt  qty=40  reorder=50  price=2.5  [LOW]
102  Wire  qty=500  reorder=100  price=12.75
103  Helmet  qty=15  reorder=20  price=350.0  [LOW]
104  Sri  qty=77  reorder=40  price=5.0

1.Add  2.Receive  3.Issue  4.View  5.Reorder  6.Exit
Choice: 5
Bolt -> order 200 = 500.0
Helmet -> order 100 = 35000.0
Total purchase cost: 35500.0


1.Add  2.Receive  3.Issue  4.View  5.Reorder  6.Exit
Choice: 6
PS C:\Users\DELL\OneDrive\Documents\PSPJ_Proj>
```

Geo Tag Photo and git hub link



git hub link-

https://github.com/bonkanpallysrinitha-coder/PSPJ-Project1.git